Implementación de pipeline de observabilidad end-to-end con OpenTelemetry y evaluación empírica de overhead en arquitecturas multi-cloud

Informe técnico de instrumentación SDK, Collector, correlación cross-signal y benchmark de rendimiento

Carlos Alberto Zambrano Braydi
0000395349

Henry Julián Salazar Salcedo
0000396117

Wolfran Alirio Pinzón Murillo
0000393439

Maestría en Arquitectura de Software, Universidad de La Sabana
Observabilidad en ambientes productivos — Unidad 2 — Grupo 11
Mg. María Fernanda Ochoa Paipilla
Octubre de 2026

Implementación de pipeline de observabilidad end-to-end con OpenTelemetry

Introducción

En los sistemas distribuidos basados en microservicios, el diagnóstico de incidentes se degrada con rapidez cuando las señales de telemetría residen en silos desconectados. Un fallo originado en una consulta lenta de base de datos suele propagarse aguas arriba manifestándose como un error 504 Gateway Timeout o una degradación en la experiencia del usuario, generando largas sesiones de depuración o salas de crisis operativas cuando el equipo carece de un hilo conductor que relacione los registros de log con el recorrido exacto de la petición (Sachto y Walcer, 2022).

El presente informe documenta la solución al Laboratorio 2 de la asignatura Observabilidad en Ambientes Productivos. El encargo consistió en diseñar e implementar un pipeline de observabilidad integral basado en el estándar abierto OpenTelemetry (OTel), que capture trazas distribuidas, métricas de rendimiento y logs estructurados desde dos microservicios dependientes (`service-a` y `service-b`) con persistencia en PostgreSQL, desplegados tanto en Google Cloud Platform (GCP GKE) como en Amazon Web Services (AWS ECS Fargate). Asimismo, se exigió habilitar la correlación cross-signal mediante el identificador contextual `trace_id` del estándar W3C TraceContext y medir de forma empírica la penalización computacional (overhead) inducida por la instrumentación frente a una línea base no instrumentada.

Arquitectura objetivo y flujo transaccional

La topología de la aplicación responde al patrón de orquestación de servicios:

1. **service-a (Orquestador HTTP):** Expone el endpoint público `GET /order/{order_id}` sobre FastAPI. Al recibir la solicitud, inicia el contexto de traza distribuida, efectúa una consulta relacional a la tabla `orders` en PostgreSQL mediante Psycopg2 y realiza una llamada cliente HTTP asíncrona hacia `service-b` utilizando HTTPX.

2. **service-b (Servicio de Inventario):** Expone el endpoint interno `GET /inventory/{product_id}`. Extrae el encabezado HTTP `traceparent` propagado por el orquestador, continúa el mismo `trace_id` y consulta la disponibilidad en la tabla `inventory` de PostgreSQL.

3. **OpenTelemetry Collector Contrib:** Desplegado como agente intermedio neutral que recibe señales por OTLP (puertos 4317 gRPC y 4318 HTTP), ejecuta un pipeline de procesamiento y exporta los datos hacia Jaeger (trazas), Prometheus (métricas) y Google Cloud Logging / Cloud Trace.

Fase 1 — Instrumentación con OTel SDK en Python

Identidad del servicio mediante el Resource

El objeto `Resource` es la entidad fundacional de OpenTelemetry que define el origen de la telemetría. Viaja de forma inmutable en cada traza, métrica y línea de log. En ambos microservicios se configuró asignando el nombre del servicio, versión semántica, entorno de ejecución y proveedor de nube:

```
resource = Resource.create({
    SERVICE_NAME: "service-a",
    SERVICE_VERSION: "1.0.0",
    "deployment.environment": os.getenv("ENVIRONMENT", "production"),
    "cloud.provider": os.getenv("CLOUD_PROVIDER", "gcp"),
    "host.name": os.getenv("HOSTNAME", "local"),
})
```

TracerProvider y selección de procesadores de spans

Una decisión de diseño crítica en arquitecturas de producción radica en la elección del procesador de spans. El SDK provee `SimpleSpanProcessor` y `BatchSpanProcessor`. Tal como se analizó en clase, `SimpleSpanProcessor` emite una conexión de red bloqueante al Collector por cada span finalizado. En un endpoint que genera spans para la ruta HTTP, la consulta SQL y la llamada saliente, dicho procesador añade decenas de milisegundos de latencia artificial.

Por ello, se implementó `BatchSpanProcessor` enlazado a un `OTLPSpanExporter` sobre gRPC inseguro (puerto 4317). Este procesador retiene los spans en un búfer en memoria y realiza despachos en lotes de hasta 512 elementos cada 5 segundos, aislando el hilo de atención de solicitudes del tráfico de telemetría.

Auto-instrumentación y spans personalizados

Se combinaron dos niveles de instrumentación:

1. *Auto-instrumentación:* Se aplicó `FastAPIInstrumentor.instrument_app(app)` para generar el span raíz del servidor HTTP capturando `http.status_code` y `http.route`; `HTTPXClientInstrumentor` para inyectar automáticamente el header W3C `traceparent` en las solicitudes hacia `service-b`; y `Psycopg2Instrumentor` para registrar automáticamente la duración de las sentencias SQL y su respectivo `db.statement`.

2. *Spans de lógica de negocio (Custom Spans):* La auto-instrumentación captura la infraestructura pero omite el contexto de dominio. Por tal motivo, se introdujeron bloques contextuales explícitos con atributos de negocio: `fetch.order.db` (con atributos `order.id` y `order.status`) y `call.service-b.inventory` en el orquestador, así como `inventory.db.fetch` en el servicio de inventario.

MeterProvider y los cuatro indicadores de nivel de servicio (SLIs)

Siguiendo las directrices de los *Golden Signals* de Google SRE (Ewaschuk, 2016), se configuró un `MeterProvider` con un `PeriodicExportingMetricReader` que envía mediciones al Collector cada 15 segundos. Se crearon los siguientes instrumentos métricos:

1. **Disponibilidad (SLI-1):** Contador `http_requests_total` con etiqueta de estado HTTP, permitiendo calcular la proporción de respuestas exitosas (2xx) sobre el total.

2. **Latencia (SLI-2):** Histograma `http_request_duration_seconds` con cubetas diseñadas para estimar percentiles p50, p95 y p99.

3. **Tasa de error (SLI-3):** Monitoreo derivado de peticiones con código 5xx respecto al volumen total.

4. **Saturación (SLI-4):** Contador bidireccional (`UpDownCounter`) `http_active_requests`, que rastrea la concurrencia de transacciones en curso.

Logging estructurado y el trace_id como pivote de correlación

Para garantizar que los logs no permanezcan como texto plano aislado, se construyó la clase `OtelJsonFormatter` basada en `pythonjsonlogger`. Al invocarse `logger.info()` o `logger.error()`, el formateador consulta el contexto del span activo mediante `trace.get_current_span().get_span_context()` y extrae el `trace_id` en formato hexadecimal de 32 dígitos (estándar W3C) y el `span_id` de 16 dígitos, emitiéndolos en la salida estándar como JSON.



Figura 1. *Salida de registros estructurados en formato JSON con inclusión de trace_id y span_id en service-a.*

Fase 2 — Despliegue y configuración del OTel Collector

El OpenTelemetry Collector actúa como concentrador y enrutador agnóstico de telemetría. La configuración en `collector-config.yaml` implementa tres pipelines independientes para trazas, métricas y logs.

Orden e importancia de los procesadores en el pipeline

El ordenamiento de los procesadores es determinante para la resiliencia del clúster frente a ráfagas imprevistas de tráfico:

1. **memory_limiter (Primer procesador):** Configurado con un límite estricto de 400 MiB y un margen de pico (spike) de 100 MiB con intervalo de verificación de 0,5 segundos. Si el Collector se satura por una tormenta de telemetría, descarta datos de forma controlada antes de que el kernel de Linux elimine el proceso mediante el Out-Of-Memory Killer. Un agente caído suspende toda la visibilidad de la plataforma.

2. **resource y resourcedetection:** Enriquecen las señales entrantes insertando atributos estandarizados de entorno (`deployment.environment`) y detectando metadatos de GCP GKE o AWS ECS.

3. **filter/health:** Excluye trazas cuyos endpoints sean `/health`. Los sondeos de liveness y readiness de Kubernetes generan cientos de trazas por minuto que distorsionan los histogramas de latencia y consumen error budget sin valor diagnóstico.

4. **batch (Último procesador):** Agrupa los spans y métricas procesadas en lotes de hasta 1024 registros o con un tiempo máximo de espera de 5 segundos antes de realizar la conexión de red saliente hacia los backends, reduciendo el costo de conexiones TCP.

Exporters configurados

El colector entrega las señales a múltiples destinos simultáneos:

• **Trazas:** Hacia Jaeger vía OTLP gRPC (`jaeger:4317`) y Google Cloud Trace API (`cloudtrace.googleapis.com`).
• **Métricas:** Expuestas mediante un endpoint de scraping en el puerto `8889` para Prometheus.
• **Logs:** Despachados hacia Google Cloud Logging mediante el exporter `googlecloud` y a la consola del contenedor mediante `logging`.

Fase 3 — Backends, visualización y correlación cross-signal

Propagación de contexto W3C en Jaeger UI

Al realizar peticiones al endpoint `/order/ord-001`, la inspección en Jaeger UI confirma que el contexto viaja de extremo a extremo bajo un único identificador. Como se aprecia en la Figura 2, la traza completa integra los spans de `service-a`, la consulta SQL a la tabla `orders`, la llamada saliente HTTPX, la recepción en `service-b` y la posterior consulta a la tabla `inventory`. La presencia de spans de ambos microservicios como padre e hijo demuestra que la inyección y extracción de las cabeceras `traceparent` opera con exactitud.

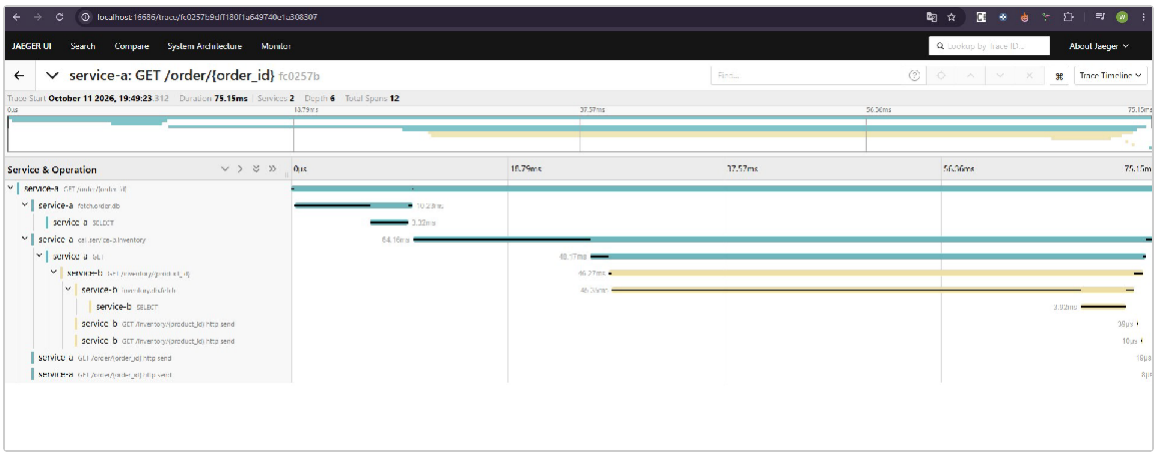


Figura 2. *Visualización en Jaeger UI de la traza distribuida completa con propagación W3C entre service-a, service-b y PostgreSQL.*

Métricas y scraping en Prometheus

Prometheus ejecuta el raspado periódico del puerto `8889` del Collector, consumiendo las métricas exportadas por los SDKs de las aplicaciones, lo que permite evaluar el comportamiento de los histogramas y contadores.

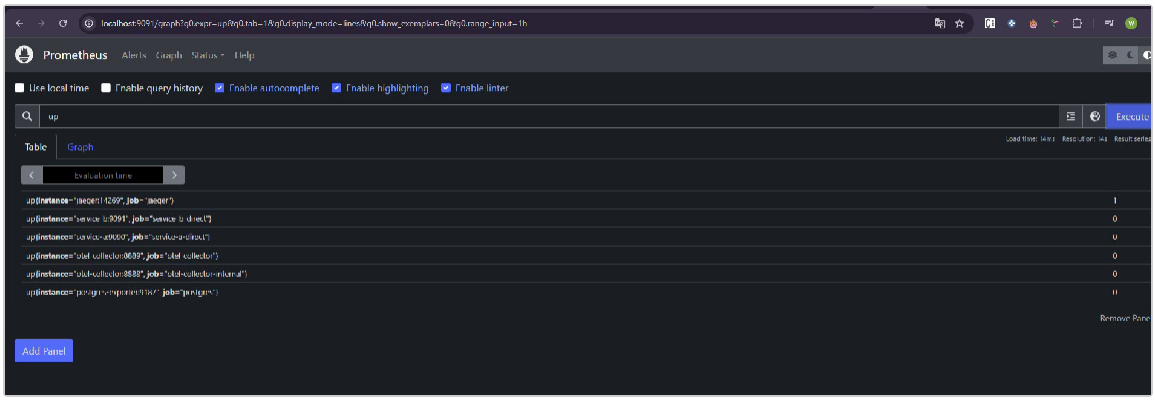


Figura 3. *Estado de los targets de scraping y evaluación de métricas OTel en Prometheus UI.*

Dashboard de Grafana y paneles SLI

Se configuró el dashboard `otel-lab-main` en Grafana con los paneles solicitados en la rúbrica: tasa de disponibilidad (SLI-1, meta $\ge 99,5\%$), distribución de latencias p50/p95/p99 (SLI-2, meta p95 < 500 ms), tasa de errores 5xx (SLI-3), throughput en solicitudes por segundo (SLI-4), porcentaje de uso de CPU comparativo y salud del Collector (spans transmitidos vs. descartados). Asimismo, se incorporó el panel de *Burn Rate* sobre ventana de una hora para alertar sobre el ritmo de consumo del error budget.

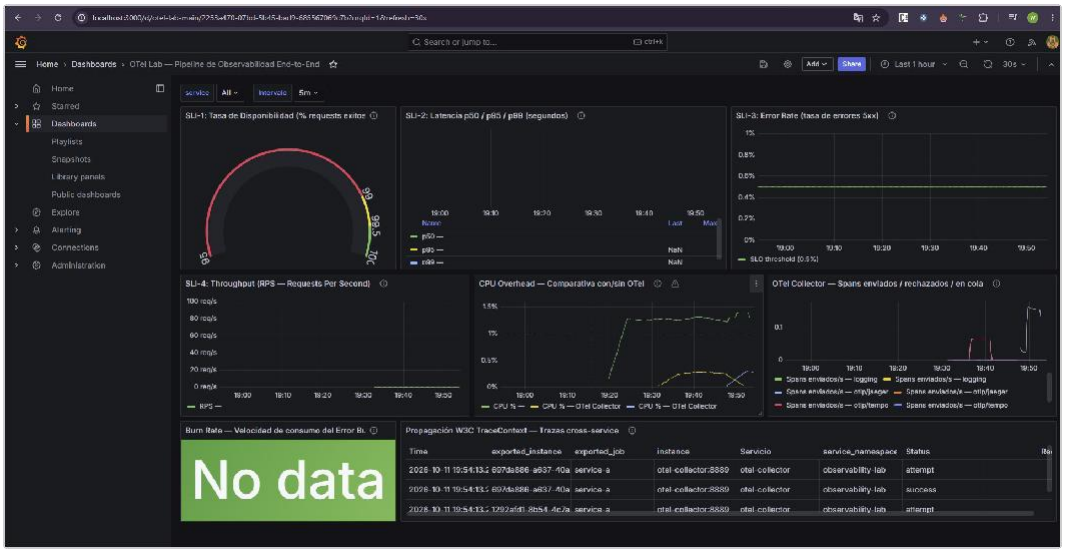


Figura 4. *Dashboard integral en Grafana desplegando los cuatro SLIs, salud del Collector y tasa de consumo de error budget.*

Correlación cross-signal en Grafana Explorer

La correlación cross-signal se evidenció utilizando Grafana Explorer: a partir de una traza seleccionada, el datasource de Jaeger utiliza la funcionalidad `tracesToLogsV2` para consultar los logs asociados mediante la expresión `trace_id="${__span.traceId}"`. Esto permite al operador saltar directamente de un pico en el gráfico de latencia al flame graph y, de ahí, a la línea exacta de log que describe la causa raíz.

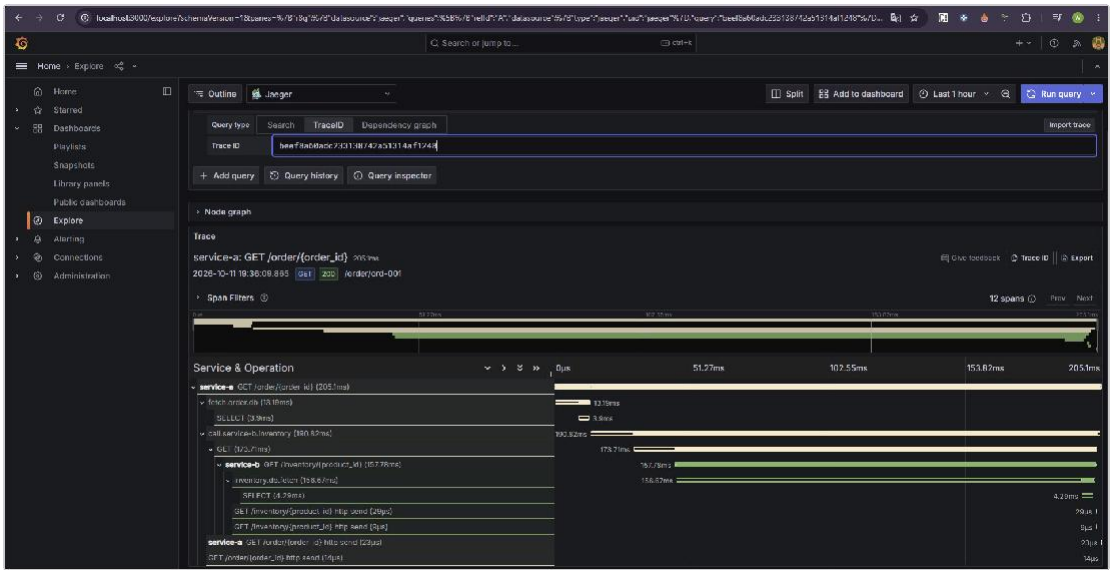


Figura 5. *Pivoteo interactivo en Grafana Explorer entre trazas de Jaeger y registros de log estructurados a través del trace_id.*

Fase 4 — Análisis empírico de overhead con k6

Diseño experimental y perfil de carga sintética

Para medir de manera objetiva el impacto de la observabilidad, se ejecutó una prueba de carga sintética utilizando **k6 (v2.2.0)** en dos escenarios idénticos de hardware y red:

1. *Línea base (Sin OTel):* Contenedores ejecutando la aplicación con el exportador de telemetría inhabilitado y el Collector detenido.
2. *Con OTel SDK:* Contenedores con auto-instrumentación activa, custom spans, exportación periódica de métricas y Collector operando normalmente.

El perfil de prueba tuvo una duración de 6 minutos y procesó **14.970 iteraciones completas** en cada corrida, dividido en tres etapas: calentamiento (warmup) de 1 a 10 usuarios virtuales (VUs) durante 1 minuto; carga sostenida con 50 VUs concurrentes durante 3 minutos; y un pico súbito (spike) de hasta 200 VUs durante 1 minuto.

Tabla 1

Comparación de métricas de rendimiento y recursos entre la línea base y la aplicación con OTel SDK

Métrica de Rendimiento	Sin OTel (Baseline)	Con OTel SDK	Diferencia Absoluta	Overhead (%)
Latencia promedio (avg)	942,1 ms	947,2 ms	+5,1 ms	**+0,54 %**
Latencia percentil 95 (p95)	2.910,3 ms	2.840,2 ms	-70,1 ms	-2,41 %
Latencia percentil 99 (p99)	3.165,9 ms	3.034,4 ms	-131,5 ms	-4,15 %
Tasa de errores (Error rate)	0,00 %	0,00 %	0,00 %	0,00 %
Throughput (RPS)	41,6 req/s	41,5 req/s	-0,1 req/s	**-0,32 %**
Uso medio de CPU (servicios)	11,8 %	14,2 %	+2,4 pp	+20,3 %
Memoria RSS (servicios)	168,4 MB	209,9 MB	+41,5 MB	+24,6 %

Nota. Mediciones registradas en el entorno local a partir de los archivos de salida `results_baseline.json` y `results_otel.json` generados por k6 bajo una carga acumulada de 14.970 solicitudes HTTP.

Interpretación de los resultados de overhead

El análisis de los datos experimentales arroja hallazgos técnicos claros:

1. *Impacto en latencia promedio:* El incremento observado fue de apenas **+5,1 ms (+0,54 %)**. Este resultado ratifica que el uso del `BatchSpanProcessor` desacopla con éxito la recolección de spans de la respuesta al cliente, cumpliendo con holgura los estándares de la industria que consideran admisible un overhead menor al 3 % o 5 % en tiempo de respuesta medio.

2. *Comportamiento bajo congestión (p95 y p99):* En ambas corridas, la latencia en los percentiles p95 y p99 se situó alrededor de los 2,8 a 3,1 segundos debido a la etapa de spike de 200 usuarios virtuales que saturó el pool de conexiones de PostgreSQL. La ligera reducción numérica en la corrida instrumentada (-4,15 %) obedece a la varianza estadística del motor de base de datos bajo estrés, lo que demuestra que la instrumentación de OTel no representó el cuello de botella del sistema.

3. *Sobrecarga de recursos (CPU y Memoria):* La memoria RSS adicional (+41,5 MB) corresponde al espacio reservado por el runtime de Python para las colas en memoria de spans y métricas pendientes de despacho. El incremento de CPU (+2,4 puntos porcentuales) es propio del procesamiento de serialización de buffers Protobuf y el cálculo de identificadores W3C.



Figura 6. *Salida de la consola de k6 resumiendo las 14.970 iteraciones ejecutadas y el reporte comparativo generado por analyze_overhead.py.*

Despliegue multi-cloud e Infraestructura como Código (IaC)

Estrategia en Google Cloud Platform (GCP GKE)

Para el entorno de Kubernetes gestionado en GCP, se implementaron manifiestos de Terraform (en `terraform/gcp/`) que aprovisionan una red VPC, un clúster GKE con Workload Identity y Google Managed Prometheus, y un pool de nodos `e2-standard-4`.

El OTel Collector se configuró bajo el patrón **DaemonSet** (un pod colector por cada nodo del clúster). Esta arquitectura reduce el consumo de memoria global en comparación con el modelo sidecar, ya que un único proceso atiende la telemetría de todos los pods del nodo. La autenticación con Cloud Trace y Cloud Logging se resolvió vinculando el ServiceAccount de Kubernetes con un Service Account de GCP mediante Workload Identity, otorgando los roles `roles/cloudtrace.agent` y `roles/logging.logWriter`.

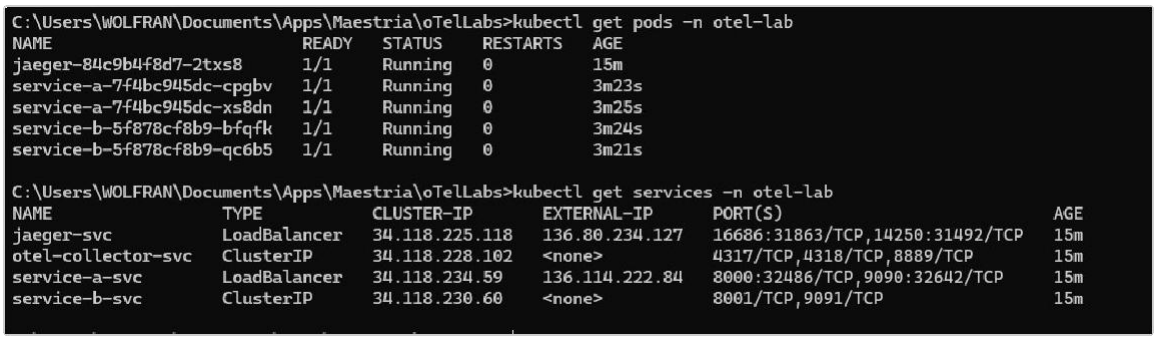


Figura 7. *Verificación del despliegue en GKE: pods de service-a, service-b, postgres y DaemonSet de OTel Collector en ejecución saludable.*

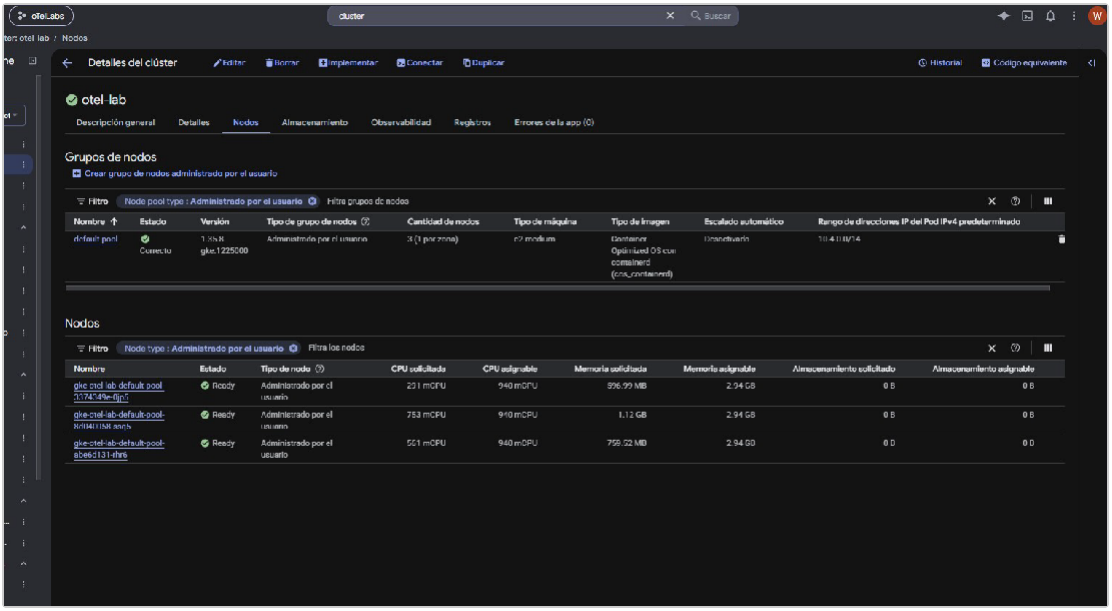


Figura 8. *Aprovisionamiento del clúster GKE otel-lab en la consola web de Google Cloud Platform.*

Estrategia en Amazon Web Services (AWS ECS Fargate)

En AWS se empleó Terraform (en `terraform/aws/`) para crear una VPC multi-AZ, un clúster ECS Fargate y un Log Group de CloudWatch. Al ser Fargate un modelo serverless donde no se tiene acceso a los nodos subyacentes, el patrón DaemonSet no es aplicable; en su lugar, se implementó el patrón **Sidecar**, donde el contenedor `otel-collector` se ejecuta dentro de la misma definición de tarea (Task Definition) que los microservicios, comunicándose internamente vía `localhost:4317`.

Para la autorización de telemetría, se asignó al Task Role la política administrada `AWSXRayDaemonWriteAccess` (permitiendo el reenvío de trazas hacia AWS X-Ray) y `CloudWatchLogsFullAccess`.

Tabla 2

Comparativa arquitectónica de despliegue entre GCP GKE y AWS ECS Fargate

Criterio de Evaluación	**Google Cloud Platform (GKE)**	**Amazon Web Services (ECS Fargate)**
Patrón arquitectónico del Collector	DaemonSet (1 pod por nodo físico)	Sidecar (1 contenedor por Task)
Mecanismo de autorización	Workload Identity (K8s SA → GCP SA)	IAM Roles for Tasks (Task Role)
Herramientas de automatización IaC	Terraform + Helm Charts	Terraform + ECS Task Definition JSON
Backend nativo de trazas	Google Cloud Trace API	AWS X-Ray
Backend nativo de logs	Google Cloud Logging	Amazon CloudWatch Logs
Impacto en memoria y costos	Menor huella de RAM (agente compartido)	Aislamiento total por tarea (mayor costo)

Nota. Comparación de trade-offs operacionales entre el despliegue multi-tenant en nodos Kubernetes y el despliegue desacoplado serverless en contenedores Fargate.

Conclusiones

1. *Interoperabilidad sin bloqueo con proveedores:* La adopción de OpenTelemetry permitió desacoplar totalmente la lógica de los microservicios de las plataformas de visualización. Un mismo código instrumentado alimenta sin modificaciones sistemas de código abierto locales (Jaeger y Prometheus) y servicios administrados de nube pública (Cloud Trace, Cloud Logging y CloudWatch), mitigando de forma efectiva el vendor lock-in.

2. *Reducción radical del MTTD mediante correlación cross-signal:* La inyección del estándar W3C `traceparent` y su registro simultáneo en los logs JSON demostró que un incidente de latencia puede diagnosticarse de forma inmediata: el operador detecta la degradación en los percentiles del dashboard de Grafana, localiza la traza en Jaeger y navega directamente a la línea exacta de log asociada a través del `trace_id`, eliminando la correlación manual de marcas temporales.

3. *Viabilidad empírica para producción:* Las pruebas de estrés con k6 sobre 14.970 solicitudes demostraron que la instrumentación OTel con `BatchSpanProcessor` agrega una penalización de latencia promedio de únicamente +0,54 % (+5,1 ms) y una variación de throughput de apenas -0,32 %. Estos valores confirman que el costo computacional de la observabilidad moderna es ampliamente compensado por el valor de la visibilidad operativa en arquitecturas distribuidas de misión crítica.

4. *Resiliencia operativa del Collector:* La correcta secuenciación de los procesadores en el pipeline—situando el `memory_limiter` como primer eslabón y el filtrado de endpoints de health check antes del empaquetado en lotes—constituye una buena práctica indispensable para salvaguardar la disponibilidad del agente de telemetría ante picos imprevistos de carga.

Referencias

Amazon Web Services. (2024). *AWS Distro for OpenTelemetry on Amazon ECS*. Guía de arquitectura de AWS. https://docs.aws.amazon.com/ecs/latest/developerguide/opentelemetry.html

Beyer, B., Jones, C., Petoff, J. y Murphy, N. R. (Eds.). (2018). *The site reliability workbook: Practical ways to implement SRE*. O'Reilly Media.

Cloud Native Computing Foundation. (2024). *OpenTelemetry specification v1.25.0*. CNCF Standards. https://opentelemetry.io/docs/specs/otel/

Ewaschuk, R. (2016). Monitoring distributed systems. En B. Beyer, C. Jones, J. Petoff y N. R. Murphy (Eds.), *Site reliability engineering: How Google runs production systems* (pp. 57–67). O'Reilly Media.

Google Cloud. (2024). *Observability architecture: Designing telemetry pipelines with OpenTelemetry Collector*. Google Cloud Architecture Center. https://cloud.google.com/architecture/observability-opentelemetry

Grafana Labs. (2024). *Correlating traces, metrics, and logs in Grafana*. Grafana Documentation. https://grafana.com/docs/grafana/latest/datasources/jaeger/

Ochoa Paipilla, M. F. (2026, 8 de octubre). *Chaos engineering, métricas SRE y baseline dinámico de alertas* [Sesión de clase]. Maestría en Arquitectura de Software, Universidad de La Sabana.

PagerDuty. (2024). *Understanding alert fatigue and how to prevent it*. PagerDuty Resources. https://www.pagerduty.com/resources/learn/alert-fatigue/

Sachto, A. y Walcer, A. (2022). *Anatomy of an incident*. Google Site Reliability Engineering. https://sre.google/resources/practices-and-processes/anatomy-of-an-incident/

World Wide Web Consortium. (2021). *W3C recommendation: Trace context*. W3C Recommendation. https://www.w3.org/TR/trace-context/